

Q&A Pod - Risk and Finance

You are defining the **Business Functional Requirements** and **Conceptual Overview** for a **Finance GPT** system, and specifically you're detailing the **Q&A Pod** module.

This Q&A Pod is not just a basic chatbot — it's a **prompt-driven AI system** with **controlled retrieval logic** that integrates **enterprise knowledge + web search + user uploads** to answer finance-related questions.

✓ Key Capabilities You Are Defining

Area	What You Intend
AI Engine	LLM-powered chatbot with structured prompt execution
Knowledge Sources	- RAG on internal knowledge base - LLM native memory - Web search via Tavily - Temporary document uploads
Retrieval Priority	1. LLM native knowledge 2. Internal KB (RAG) 3. Web search fallback 4. Uploaded Documents
Response Style	Structured response including: ✓ Interpretation of user query ✓ Source transparency ✓ Gaps identified ✓ Final answer
Conversation Memory	Maintain context while managing token limits
Use Case Coverage	Type 1: General finance questions- Type 2: BMO internal policy/application questions- Type 3: Peer comparison (external market data)

Area	What You Intend
Tools integrated	✓ RAG pipeline ✓ Web search (Tavily) ✓ File Upload & parsing ✓ Context-driven prompt orchestration

✓ Purpose & Business Fit

This Q&A Pod aligns directly to real finance workflows:

- ✓ Operational queries
- ✓ Policy interpretation
- ✓ Peer benchmarking
- ✓ Knowledge reuse across teams

Orchestrator

User Prompt

↓

Routing Logic (Prompt Interpreter)

↓

Choose tool(s):

- |— RAG Search Tool (internal KB)
- |— Tavily Web Tool
- |— File Upload Tool
- |— LLM Native Response

↓

Answer Composer (Final structured response)

✓ Router Tool

system = ""

You are a routing controller for Finance GPT. Your job is to decide which

knowledge source(s) should be used to answer a user query based on priority and completeness.

You must return a JSON object with:

```
{
  "primary_tool": "...",
  "fallback_tools": [],
  "reason": "...",
  "needs_multiple_sources": true/false
}
```

Routing PRIORITY (always follow this order):

1. "llm_only" - If the question is general finance AND does not need BMO-specific info
2. "rag_internal" - Use BMO internal knowledge base (SharePoint, internal KB, policies)
3. "web_search" - Use Tavily search for peer comparison or external facts
4. "uploaded_docs" - Use user-provided documents if query refers to them

MULTI-SOURCE RULE:

If one tool cannot fully answer a question, route additional tools in order of priority.

EXAMPLES:

- "What is net present value?" → llm_only
- "What is BMO's credit risk policy?" → rag_internal
- "Compare BMO liquidity ratios with RBC" → rag_internal + web_search
- "Summarize this document I uploaded" → uploaded_docs

Example router output:

```
{
  "primary_tool": "rag_internal",
```

```
"fallback_tools": ["web_search"],  
"reason": "Internal policy answer found but peer comparison needed for  
completeness",  
"needs_multiple_sources": true  
}
```

✓ Standard JSON Output Format (Finalized)

Every tool will return output using this structure:

```
{  
  "tool_name": "",  
  "response": "",  
  "sources": [],  
  "metadata": {}  
}
```

✓ Consistency Rules for All Tools

- ✓ Always include `tool_name`
- ✓ Always include `response`
- ✓ Always include `sources` (empty list if none)
- ✓ `metadata` reserved for tool-specific info
- ✓ No hallucinated sources allowed
- ✓ Clean JSON only — no natural language outside fields

✓ RAG Tool – Inputs & Outputs

Input Parameter	Purpose
context	Sub-category inside department domain (e.g., "treasury", "capital markets")
department	Top-level knowledge area (e.g., "finance", "risk")
prompt_template	Configuration-based search prompt
user_query	Actual user question
top_k	Number of chunks to retrieve

✓ Context + Department → Used to select the **right vector DB namespace**

✓ Prompt → Controls **semantic search behavior**

✓ Top-k → Controls **retrieval depth**

✓ Output of RAG Tool

Output Field	Purpose
response	Generated answer using retrieved context
sources	List of document chunk names + page numbers
retrieved_chunks	Raw text chunks used for answer

✓ Confirmed Tool Signature

In Python pseudocode:

```
{  
"tool_name": "rag_internal",
```

```

"response": "",
"sources": [],
"metadata": {
  "department": "",
  "context": "",
  "top_k": ""
}
}

```

Tavily Web Search Tool

Input Parameters

Name	Type	Required	Description
<code>query</code>	string		User query passed into search step
<code>search_prompt</code>	string		Admin-configured prompt that guides how to structure the search query
<code>max_results</code>	int		Number of search results to retrieve (default = 5)

```

{
  "tool_name": "web_search",
  "response": "",
  "sources": [],
  "metadata": {}
}

```

```
{
  "tool_name": "web_search",
  "response": "According to Reuters, RBC reported net income of $3.9B in Q3 2024.",
  "sources": [
    { "url": "https://www.reuters.com/article/rbc-earnings..." }
  ],
  "metadata": {
    "query": "RBC net income Q3 2024",
    "search_prompt": "Search only trusted financial news sources like Reuters, Bloomberg, or Yahoo Finance.",
    "max_results": 3
  }
}
```

Finalized Uploaded Documents Tool

Inputs

Name	Type	Required	Description
<code>file_ids</code>	array<string>		Storage IDs for uploaded files (S3 or local paths)
<code>user_query</code>	string		User query
<code>append_prompt</code>	string		Configurable admin prompt guiding how to use uploaded docs
<code>max_chars</code>	int		Max characters to include from uploaded document(s)

Output (Standard JSON)

```
{
  "tool_name": "uploaded_docs",
```

```
"response": "",
"sources": [],
"metadata": {}
}
```

Example Output

```
{
  "tool_name": "uploaded_docs",
  "response": "Based on the uploaded policy, IFRS 9 expected credit loss uses PD, LGD, and EAD as core components.",
  "sources": [
    { "file_name": "IFRS9_Overview.pdf" }
  ],
  "metadata": {
    "max_chars": 30000,
    "truncated": false
  }
}
```

Answer Composer (Final Response Contract)

Make the final output consistent regardless of which tools ran.

Final Response JSON

```
{
  "answer": "",
  "sources": [],
```



```

"reasoning_summary": "",
"gaps": [],
"follow_ups": []
}

```

- **answer** : the concise user-facing response
- **sources** : list of { "name": "LiquidityPolicy_2024.pdf" } or { "url": "https://..." }
- **reasoning_summary** : 1–3 sentences: "what we did & why" (auditable)
- **gaps** : missing data or assumptions
- **follow_ups** : suggested clarifying questions/actions

2) Orchestration Graph (LangGraph skeleton)

Define a minimal, reliable graph with deterministic edges.

```

# nodes: router → executor → composer
# memory: conversation buffer (token-aware truncation)

def router_node(state):
    # returns routing plan (primary_tool, fallback_tools, needs_multiple_sources)
    return route(state["user_query"], state["context"])

def executor_node(state):
    plan = state["routing_plan"]
    results = []

    def run_tool(tool_name):
        # map to callable + standard inputs taken from state/config
        out = TOOLS[tool_name](**state["tool_inputs"][tool_name])

```

```

        results.append(out)

    run_tool(plan["primary_tool"])

    if plan.get("needs_multiple_sources"):
        for t in plan.get("fallback_tools", []):
            run_tool(t)

    state["tool_results"] = results
    return state

def composer_node(state):
    # merge tool_results → final response
    final = compose_answer(state["user_query"], state["tool_results"])
    state["final_response"] = final
    return state

```

3) Tool Contracts (you already set these)

- **RAG** (stateless): `department, context, prompt_template, user_query, top_k` → standard tool JSON
- **Web (Tavily)** (stateless): `query, search_prompt, max_results` → standard tool JSON
- **Uploaded Docs (Append Mode)** (stateless): `file_ids, user_query, append_prompt, max_chars` → standard tool JSON
- **LLM Native**: no tool required; the agent answers directly when router picks `llm_only`.

4) Admin Config Schema (YAML / JSON)

Drive behavior without code changes.

yaml

 Copy

```
departments:
  finance:
    contexts:
      treasury:
        rag:
          top_k: 4
          prompt_template: "Use only canonical treasury policy docs..."
router:
  search_prompt: "Use reputable finance sources (Reuters, Bloomberg)..."
  allow_multi_source: true
limits:
  max_chars_upload_append: 40000
  web_max_results: 5
response:
  style: "concise|detailed"
```

6) Token & Cost Controls

6.1 Token-aware conversation buffer

Maintain a rolling history; drop oldest messages before passing to the LLM.

```
import tiktoken

def prune_history(messages, model, max_tokens):
    enc = tiktoken.encoding_for_model(model)
```

```

def count(msgs): return sum(len(enc.encode(m["content"])) for m in msgs)
while count(messages) > max_tokens:
    # drop the oldest user/assistant exchange (keep system + latest user)
    for i, m in enumerate(messages):
        if m["role"] in ("user","assistant"):
            del messages[i]; break
    return messages

# usage pre-LLM call
messages = prune_history(messages, model="gpt-4o-mini", max_tokens=cfg
["limits"]["llm_max_tokens"])

```

Keep **system** prompt and **most recent** turns; discard older.

6.6 Observability (recommended)

Emit structured logs for every run:

- router decision,
- tool inputs (types & IDs; never raw text),
- gaps list,
- final sources list.

Use OpenTelemetry or JSON logs:

```

log.info("run", extra={
    "query": state["user_query"],
    "routing_plan": plan,
    "tool_results_meta": [r["tool_name"] for r in state["tool_results"]],
    "gaps": state["gaps"],
    "sources": [s for r in state["tool_results"] for s in r.get("sources",[])],
})

```

5.2 Per-tool timeouts

Wrap each tool invocation with a timeout and error mapping. Skip the failed tool, record a gap, continue.

```
import concurrent.futures as cf

def run_with_timeout(fn, kwargs, seconds, tool_name, state):
    try:
        with cf.ThreadPoolExecutor(max_workers=1) as ex:
            fut = ex.submit(fn, **kwargs)
            return fut.result(timeout=seconds)
    except cf.TimeoutError:
        state["gaps"].append(gap("timeout", f"{tool_name} exceeded {seconds} s", tool_name))
        return None
    except Exception as e:
        state["gaps"].append(gap("tool_error", f"{tool_name} error: {e}", tool_name))
        return None
```

Config-driven:

```
seconds = cfg["limits"].get("tool_timeout_seconds", 8)
out = run_with_timeout(TOOLS[name], inputs, seconds, name, state)
```

Error Handling & Fallbacks

5.1 Structured “gaps” model

Use a consistent structure to record issues so the **composer** can surface them.

```
def gap(kind: str, detail: str, tool_name: str = None):  
    return {"kind": kind, "detail": detail, "tool": tool_name}
```

Common kinds:

- "timeout", "empty_result", "truncated_upload", "conflict_external_vs_internal", "tool_error"

Your run state carries `gaps: List[dict]`:

```
state = {"user_query": q, "tool_results": [], "gaps": []}
```

Evaluation & QA

- **Golden set:** 25–50 curated questions across T1/T2/T3 with expected citations.
- **Automated evals:** Offline harness to run router → tools → composer, track pass/fail.
- **Prompt regression tests:** Snapshot prompts; diff on PR; fail build on degradation.
- **Manual review rubric:** 1–5 scoring for accuracy, citation validity, clarity; weekly sample.